Anteater Chess Game (Version 1.2)

Software Specification

By Thoughtless Airheads

Members: Sanaa Bebal, Ava Chinn, Alonso De La Cruz, Estrella Distancia Zamora, Lauren Howe, Samuel Kim, and David Lee

Created for the class EECS 22L at University of California, Irvine

Table of Contents

Table of Contents 2
Glossary 3
- Game 3
- Pieces 3
- Moves 4

1. Software Architecture Overview 5
- 1.1 Main Data Types and Structures 5
- 1.2 Major Software Components 6
- 1.3 Module Interfaces 6
- 1.4 Overall Program Control Flow 11

2. Installation 12
- 2.1 System requirements and compatibility 12
- 2.2 Setup and configuration 12
- 2.3 Building, compilation, and installation 12

3. Documentation of Packages, Modules, and Interfaces 13
- 3.1 Detailed Description of Data Structures 13
- 3.2 Detailed Description of Functions and Parameters 15
- 3.3 Detailed Description of Input and Output Formats 21
  - Syntax for Making a Move (Text-Based Version) 21
  - Format of Moves Recorded in User Log (Algebraic Notation) 22

4. Development Timeline 22
- 4.1 Partitioning of Tasks 22
- 4.2 Team Member Responsibilities 23

5. References and External Libraries 23
Contact Information 23
Terms and Conditions 24
- Copyright 24
- Disclaimer of Warranty 24

Index 24

Glossary

(of terms used in the implementation)

Game

- **Anteater Chess**–a variation of the game chess in which pawns are replaced by pieces named “ants” and players have additional pieces named “anteaters”; the board configuration and starting layouts are changed in response to these changes (see “Ranks”, “Files”, and “Anteater”)
- **Board**–the interface through which the game is played; rectangular setup of squares that can be occupied by pieces; size determined by the number of ranks and files (see “Ranks” and “Files”)
- **Check**–one player is capable of capturing their opponent’s king on the player’s next move, but the opponent is capable of making a move such that their king cannot be captured during the player’s next move
- **Checkmate**–one player is guaranteed to have their king captured during the opponent’s next move, regardless of their own move; this ends the game, with the aforementioned opponent then being declared the winner of the game
- **Files**–the columns of the board, labeled from the left side of the board to the right; while there are eight (labeled A through H) files in normal chess, there are 10 (labeled A through J) files in anteater chess
- **Piece**–a general term used to refer to a(n) ant, anteater, bishop, king, knight, queen, or rook
- **Ranks**–the rows of the board, numbered from the bottom of the board to the top; there are eight (numbered 1 through 8) ranks in both normal chess and anteater chess
- **Turn**–a player moves one piece to a desired, legal position; turns alternate between players, starting with the player using the white chess set

Pieces

- **Ant**–the equivalent of a pawn in a normal chess game; usually moves one square forward at a time, but can also move two squares forward at a time if it is the very first move performed by a player in the game, and can move diagonally to capture pieces (see “Capture”)
    - Each player has 10 of these at the beginning of the game
- **Anteater**–can move one square in any direction like a king; cannot capture any pieces except ants (i.e. not a threat to the opposing king), but can capture any and all ants adjacent to the a captured ant, provided that they are connected vertically or horizontally (see “Ant eating”); does not have an equivalent in a normal chess game
    - Each player has two of these at the beginning of the game
- **Bishop**–behaves the same as a bishop in a regular chess game; can move diagonally along an unobstructed path
    - Each player has two of these at the beginning of the game

- **King**–behaves the same as a king in a regular chess game; can move one square in any direction, but can never move into check (see “Check”)
    - Each player has one of these at the beginning of the game
- **Knight**–behaves the same as a knight in a regular chess game; can move two squares in a straight line and then one square in a perpendicular direction; can “jump” over pieces (i.e. does not require an unobstructed path)
    - Does not capture pieces it “jumps” over
    - Each player has two of these at the beginning of the game
- **Queen**–behaves the same as a queen in a regular chess game; can move in any direction (horizontally, vertically, and diagonally) along an unobstructed path
    - Each player has one of these at the beginning of the game
- **Rook**–behaves the same as a rook in a regular chess game; can move in a straight line (horizontally and vertically) along an unobstructed path
    - Each player has two of these at the beginning of the game

Moves

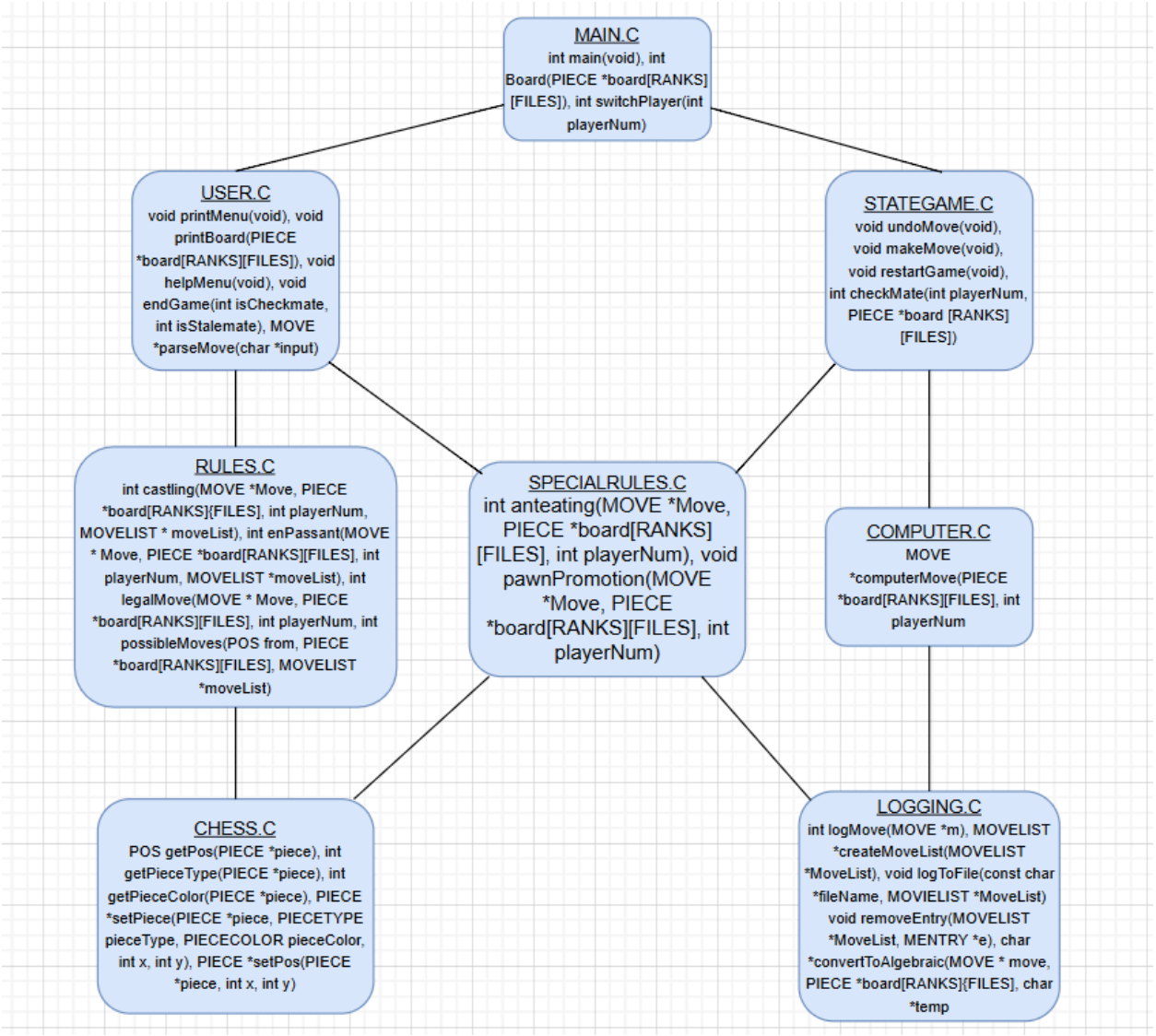
- **Ant eating**—special capture move that can be performed by the anteater piece (see “Anteater”); can capture any and all ants adjacent to a captured ant, provided they are connected vertically or horizontally; anteater piece ends in the position of the last eaten ant, with the order of “eating” possibly up to the player
    - Note: Player can choose to not “eat” all reachable ants during their move
- **Capture**–the player moves a piece to the position of an opponent’s piece (except for en passant; see “En Passant”) and takes the opponent’s piece off the board; the removed piece is then considered out of play
- **Castling**—There are two conditions to use castling. First, King and Rook should be in their original position(must have not moved before). Second, no piece can be between King and Rook. Third, the King must not currently be in check. Fourth, the King cannot pass through or land on a square that is under attack. Once all conditions are satisfied, the player can move King two squares towards Rook, and then place Rook on the square next to the king on the opposite side.
- **En passant**—A pawn capture that can occur when an opponent’s pawn moves two squares forward from its starting position and lands beside your pawn. If your pawn is on the fifth rank (for White) or fourth rank (for Black), you may capture the opponent’s pawn as if it had only moved one square forward. This capture must be made immediately on the next move, otherwise the opportunity is lost.
- **(Moving) backward**–a piece is moved towards its team’s starting position on the board
- **(Moving) forward**–a piece is moved away from its team’s starting position on the board
- **Pawn promotion**–any pawn piece that reaches the rank furthest from their starting position immediately exchanges their pawn for a Queen, Rook, Bishop, Knight, or Anteater; the new piece does not have to be one that was previously captured

1. Software Architecture Overview

1.1 Main Data Types and Structures

- struct Pos (typedef POS): Represents a position on the anteater chess board
    - Stores 2 values:
        - int x: stores x coordinates of position
        - int y: stores y coordinates of position
- struct Piece (typedef PIECE): Represents a game piece
    - Stores 3 values:
        - int pieceColor: stores color of piece (black or white), represented by PIECECOLOR enumeration (see Section 3.1)
        - int pieceType: stores type of piece (king, queen, etc.), represented by PIECETYPE enumeration (see Section 3.1)
        - POS pos: stores location of piece on the board
- struct Move (typedef: MOVE): Represents a possible or executed move
    - Stores 2 values:
        - POS pos1: Starting position of a piece at the beginning of a turn
        - POS pos2: Ending position of a piece at the end of a turn
- struct MoveList (typedef: MOVELIST): Stores possible moves or executed moves in a doubly-linked list
    - Stores 3 values:
        - MENTRY *first: First entry in the list
        - MENTRY *last: Last entry in the list
        - int length: Length of the list
- struct Mentry (typedef: MENTRY): Represents an entry in MOVELIST
    - Stores 3 values:
        - MENTRY *prev: Previous entry in the list (or NULL if it is the first entry)
        - MENTRY *next: Next entry in the list (or NULL if it is the last entry)
        - MOVE *move: Move associated with this entry in the list

1.2 Major Software Components



1.3 Module Interfaces

Major functions in each module

Main.c (running the actual game & calls on the other files)
Local files required: Main.h, User.h, Rules.h, Chess.h, StateGame.h, Computer.h, Logging.h
Libraries required: stdio.h, stdlib.h

- int main(void) - runs the main program
    - Calls: initBoard(), switchPlayer(), printMenu(), printBoard, helpMenu(), endGame, parseMove(), legalMove(), checkMate(), staleMate(), undoMove(),

makeMove(), restartGame(), computerMove(), createMoveList(), deleteMoveList(), logToFile()

- void initBoard(PIECE *board[RANKS][FILES])- provides a clean board for the game
  - Calls: N/A
- int switchPlayer(int playerNum) - toggles between player 1 and player 2
  - Calls: N/A

User.c (user interface)

Local files required: User.h, Chess.c

Libraries required: stdio.h and/or X11/Xlib.h (depends on version), string.h

- void printMenu(void) - prints (or displays, if GUI) main menu
  - Calls: N/A
- void printBoard(PIECE *board[RANKS][FILES]) - prints out (or displays, if GUI) the current state of the board
  - Calls: N/A
- void helpMenu(void) - scroll down text that explains how to play and use the program
  - Calls: N/A
- void endGame(int isCheckmate, int isStalemate) - creates a display indicating a win/loss or stalemate (would most likely be based on just saying “player 1 / player 2 wins” for win/loss case, as saying whether the user won would only be applicable for one of the three options and would require more inputs)
  - Note: checkMate() returns 0, 1, or 2; staleMate() returns 0 or 1
  - Calls: N/A
- MOVE parseMove(char *inputStr) - takes user input and returns move based on user input
  - Note: Input will depend on whether the text interface or the GUI is active; for the text version it will likely be a string. *For the GUI, we would divide the board into cells and use mouse-click detection and mouse-position functions to decipher player input. This would effectively remove the need for MOVE parseMove(char *inputStr).
  - Calls: N/A

Rules.c (defining the moves of the games)

Local files required: User.h, Computer.h, StateGame.h, Chess.h

Libraries required: stdio.h

- int castling (MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList) - returns 1 when conditions are fulfilled, allowing the king to move 2 positions towards the rook, and the rook to hop over the adjacent square
  - Calls: inCheck(), getPieceType(), getPieceColor()
- int enPassant (MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList) - returns 1 when conditions are fulfilled, allowing a pawn to take an opposing pawn that has moved two positions, move needs to be made immediately on the next turn
  - Calls: getPieceType(), getPieceColor()

- int legalMove(MOVE *Move, PIECE *board[RANKS][FILES], int playerNum, MOVELIST *MoveList) - based on the current board and the proposed move, returns 0 if the proposed move is illegal and 1 if it is legal
    - Calls: Moveable(), castling(), enPassant(), inCheck()
- int Moveable(MOVE *Move, PIECE *board[RANKS][FILES]) - returns 1 if the Move is within the specific piece's ability and 0 if the move isn't. This considers whether the terminal position is on a cell with another piece of the same color
    - Calls: getPieceType(), getPieceColor()

- MOVELIST *possibleMoves(POS from, PIECE *board[RANKS][FILES], MOVELIST *MoveList, int playerNum) - analyzes piece type, color, and position and determines a linked list containing possible moves
    - Calls: legalMove()

SpecialRules.c (defining moves that require further player input)
Local files required: Rules.h, Chess.h
Libraries required: stdio.h

- int anteating(MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList) - If an anteater captures a pawn, allow the user to capture adjacent pawns in a single rank or file.
    - Calls: printBoard(). setPos(), setPiece(), getPieceType(), getPieceColor()
- void pawnPromotion(MOVE *Move, PIECE *board[RANKS][FILES]) - If a pawn reaches a position on the opposite side of the board (rank 8 for white, rank 1 for black), prompts the user to choose one of the following pieces: queen, bishop, rook, knight, anteater.
    - Calls: setPiece(), getPieceType()

Chess.c (getter & setters for the data types of each piece on the board)
Local files required: User.c
Libraries required:

- POS getPos(PIECE *piece) - returns the position (x and y coordinates) of a piece
    - Calls: N/A
- int getPieceType(PIECE *piece) - returns the type of a piece (king, queen, anteater, etc.), represented as an integer
    - Calls: N/A
- int getPieceColor(PIECE *piece) - returns the color of a piece (black, white), represented as an integer
    - Calls: N/A
- PIECE *setPiece(PIECE *piece, PIECETYPE pieceType, PIECECOLOR pieceColor, int x, int y) - sets all values of a piece (useful for initialization)
    - Calls: N/A
- PIECE *setPos(PIECE *piece, int x, int y) - sets the position of a piece (useful for moving pieces); uses a default position not on the board to indicate that a piece has been captured

StateGame.c
Local files required: User.h, Computer.h, Logging.h, Rules.h, Main.h
Libraries required: stdio.h

- void undoMove(void) - reversing the last chess move from the game, restoring pieces to their previous positions, replacing captured pieces, and any other special rules
    - Calls: removeEntry()
- void makeMove(int playerNum, int isComputer, PIECE *board[RANKS][FILES], MOVELIST *logMoveList) - Takes in user input, updates the board state by moving pieces from start to desired location, handling captured pieces, and other special circumstances. If it detects an improper input (illegal move or improper format), it reprompts the user.
    - Calls: legalMove(), castling(), enPassant(), computerMove(), possibleMoves(), deleteMoveList(), logMove()
- void restartGame(void) - resetting the board, clearing out logged moves, the starting player as White, and updating the UI state
    - Calls: deleteMoveList(), initBoard()
- int checkMate(int playerNum, PIECE *board[RANKS][FILES]) - returns 0 if neither player is in checkmate, 1 if player one is in checkmate, and 2 if player two is in checkmate
    - Calls: legalMove()
- int staleMate(int playerNum, PIECE *board[RANKS][FILES], MOVELIST *MoveList) - returns 0 if game has not reached stalemate and 1 if it has
    - Calls: possibleMoves()
- int inCheck(int playerNum, PIECE *board[RANKS][FILES]) - returns 0 if player playerNum is not in check and 1 if it is in check
    - Calls: N/A

Computer.c
Local files required: Rules.h, SpecialRules.c, Chess.c
Libraries required: stdio.h

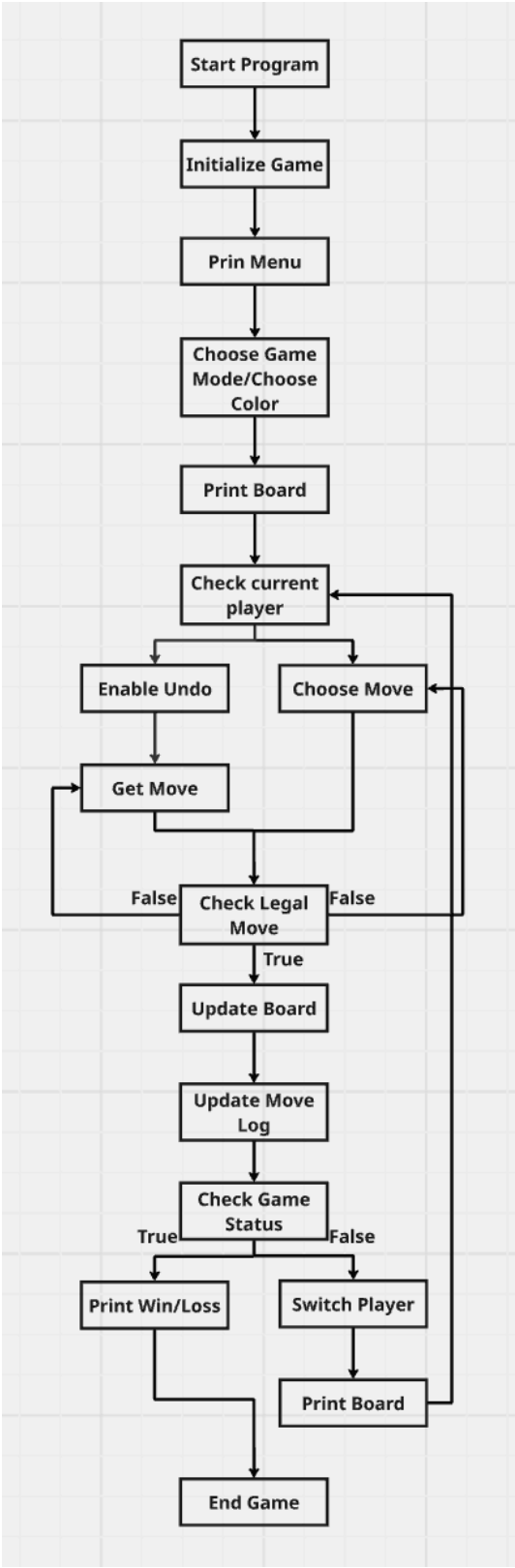
- MOVE computerMove(PIECE *board[RANKS][FILES], int playerNum) - using a Simplified Evaluation Function or a point-based system in order for the computer to provide better possible moves
    - Calls: legalMove(), possibleMoves()

Logging.c
Local files required: StateGame.h
Libraries required: stdio.h, stdlib.h, string.h

- int logMove(MOVE *m) - adds the specified move to the list
    - Calls: N/A
- MOVELIST *createMoveList(MOVELIST *MoveList) - creating new lists for current game when starting the game
    - Calls: N/A
- void deleteMoveList(MOVELIST *MoveList) - when restarting the game, clearing out the move lists for both Black and White

    - Calls: removeEntry()
- void logToFile(const char *fileName, MOVELIST *MoveList) - takes the list containing all the moves of the game and inputs all the made moves into a readable file
    - Calls: N/A
- void removeEntry(MOVELIST *MoveList, MENTRY *e) - when the undo function is called, removing the last entry within the list
    - Calls: N/A
- char *convertToAlgebraic(Move *move, PIECE *board[RANKS][FILES], char *temp) - converts a player input into algebraic format
    - Calls: getPieceType()

1.4 Overall Program Control Flow



2. Installation

2.1 System requirements and compatibility

The user must satisfy the following conditions to correctly install the software:

- **Operating System:** RHEL8 64-bit Linux
- **Compiler:** GNU Compiler Collection (gcc)
- **Build System:** GNU *make*
- **Libraries:** Standard C libraries
- **Utilities:** tar, gzip, evince, and shell environment
- **Hardware Requirements:** Minimal CPU and memory resources sufficient for compiling and executing.

2.2 Setup and configuration

Using the command line, go into the installed folder and run the following code:

A. **Extract the Source Package**
```
gtar xvzf Chess_V1.0_src.tar.gz
cd chess
```
B. **Build the Software**
```
cd src
make
```
C. **Run the Executable**
```
cd ../bin
./chess
```
D. **Remove compiled object files**
```
cd ../src
make clean
```

2.3 Building, compilation, and installation

A. **Create the Binary Distribution**
```
tar -zcvf Chess_V1.0.tar.gz \
        ./doc/README  \
        ./doc/COPYRIGHT  \
        ./doc/Chess_UserManual.pdf  \
        ./doc/INSTALL  \
        ./bin/chess  \
```

./bin/*.bmp \
./Makefile

B. **Create the Source Distribution**

tar -zcvf Chess_V1.0_src.tar.gz \
./doc \
./bin \
./src \
./Makefile

C. **Purpose of Each Package**

Chess_V1.0.tar.gz - Contains the executable and documentation for end users
Chess_V1.0_src.tar.gz - Contains full source code for developers and maintainers

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed Description of Data Structures

Critical snippets of source code

(Will be in Chess.h as all the files depend upon these definitions)

```
struct Pos{
    int x;
    int y;
};
typedef struct Pos POS;


enum PieceColor{
    White,
    Black
};
typedef enum PieceColor PIECECOLOR;

enum PieceType{
    King,
    Queen,
    Anteater,
```

```
    Bishop,
    Rook,
    Knight,
    Ant
};
typedef enum PieceType PIECETYPE;

struct Piece{
    PIECECOLOR PieceColor;
    PIECETYPE PieceType;
    POS pos;
};
typedef struct Piece PIECE;


struct Move{
    POS pos1;
    POS pos2;
};
typedef struct Move MOVE;


struct Mentry{
    MENTRY *prev;
    MENTRY *next;
    MOVE *move;
};
typedef struct Mentry MENTRY;


struct MoveList{
    MENTRY *first;
    MENTRY *last;
    int length;
};
typedef struct MoveList MOVELIST;
```

3.2 Detailed Description of Functions and Parameters

main.c - entry point and entering the game loop
Provides: program entry and the top-most level game loop
Requires: libUser.a, libStateGame.a, libRules.a, libLogging.a, libComputer.a

int main(void)
- Arguments: none
- Result:
    - 0 on clean exit

void initBoard(PIECE *board[RANKS][FILES])
Arguments:
- board: the board array that needs to be initialized

Result:
- none: populates board with pieces in their starting positions

int switchPlayer(int playerNum)
Arguments:
- playerNum: current player's number/indicator of 0 for White or 1 for Black

Result:
- the other number

User.c - user interface
Provides: all the displays and input parsing done for text and GUI
Requires: chess.h

void printMenu(void)
Arguments: none
Result: none

void printBoard(PIECE *board[RANKS][FILES])
Arguments:
- The 2D array board, which contains

Result:
- Current state of board

void helpMenu(void)
Arguments: none
Results:none

void endGame(int isCheckmate, int isStalemate)

Arguments:
- isCheckmate: checks if players are in checkmate
- isStalemate: check if players are in stalemate (draw)

Results:
- Ends the game depending on if a player won or if it ended in a draw

Rules.c - parameters for how pieces move on the board
Provides: generation of piece movement, checking if move is legal
Requires: Chess.h, libSpecialRule.c

<u>int castling(MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList)</u>
Arguments:
- Move: pointer to the intended move of a specific piece
- board: current state of the board
- MoveList: storing collection of generated legal moves

Result:
- Indicates whether king and chosen rook are successfully moved

<u>int enPassant(MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList)</u>
Arguments:
- Move: pointer to the intended move of a specific piece
- board: current state of the board
- MoveList: storing collection of generated legal moves

Result:
- Indicates whether a valid en passant capture is possible by returning either 0 or 1 for not legal and legal, respectively

<u>int legalMove(MOVE *Move, PIECE *board[RANKS][FILES], int playerNum, MOVELIST *MoveList)</u>
Arguments:
- Move: pointer for the intended move of a specific piece
- board: current state of the board
- playerNum: 0 for White, 1 for Black, whose turn it is
- MoveList: storing collection of generated legal moves

Result:
- Returns 1 if the proposed move is legal
- Returns 0 if the proposed move is illegal

<u>int Moveable(MOVE *Move, PIECE *board[RANKS][FILES])</u>
Arguments:
- Move: pointer for the intended move for the specific piece
- board: current state of the board

Result:
- Returns 1 if the move is within the specific piece's ability

- Returns 0 if there's a piece of the same color on the terminal move position
- Returns 0 if the move transcends what's possible for the specific piece

<u>MOVELIST *possibleMoves(POS from, PIECE *board[RANKS][FILES], MOVELIST *MoveList, int playerNum)</u>
Arguments:
- board: current state of the board
- MoveList: storing collection of generated legal moves
- from: starting position of a given piece
- playerNum: 0 for White, 1 for Black, whose turn it is

Result:
- Returns a referenced list of all legal moves for a specific piece on the board.

SpecialRules.c - parameters for special moves
Provides: more complex generation of piece movement, checks if move is legal
Requires: Chess.h, libRules.a

<u>int anteating(MOVE *Move, PIECE *board[RANKS][FILES], MOVELIST *MoveList)</u>
Arguments:
- Move: pointer to the intended move of a specific piece
- board: current state of the board
- MoveList: storing collection of generated legal moves

Result:
- 1: special move completed
- 0: no additional capture or move performed / failed / invalid continuation

<u>void pawnPromotion(MOVE *Move, PIECE *board[RANKS][FILES])</u>
Arguments:
- Move: pointer to the intended move of a specific piece
- board: current state of the board

Result:
- Pawn if replaced with the selected promoted piece, board updated

Chess.c - gets the data types of each piece on the board
Provides: getters and setters for the PIECE struct
Requires: Chess.h

<u>POS getPos(PIECE *piece)</u>
Arguments:
- piece: pointer to specified piece

Result:
- position of a piece in x and y coordinates

int getPieceType(PIECE *piece)
Arguments:
- piece: pointer to specified piece

Result:
- Integer of corresponding piece type

int getPieceColor(PIECE *piece)
Arguments:
- piece: pointer to specified piece

Result:
- Integer of corresponding player color

PIECE *setPiece(PIECE *piece, PIECETYPE pieceType, PIECECOLOR pieceColor, int x, int y)
Arguments:
- Piece: pointer to specified piece
- pieceType: pointer to specified type
- pieceColor: pointer to specified color
- x: pointer to specified x-coordinate
- y: pointer to specified y-coordinate

Result:
- New piece including specified type, color, and location

PIECE *setPos(PIECE *piece, int x, int y)
Arguments:
- piece: pointer to specified piece
- X: pointer to specified x-coordinate
- Y: pointer to specified y-coordinate

Result:
- Moves piece to new position

StateGame.c - handles the current state of the board and game status
Provides: initialization of the game board, undoing the game, resetting the game, has the game been won
Requires: Chess.h, libRules.a , libLogging.a, libChess.a

void undoMove(void)
Arguments: none
Result:
- Previous state of board

void makeMove(int playerNum, int isComputer, PIECE *board[RANKS][FILES], MOVELIST *logMoveList)
Arguments:

- playerNum: player number (0 or 1)
- isComputer: specifies whether computer is making move (1) or user is making move (0)
- board: 2D array of pointers to pieces in current positions
- logMoveList: pointer to list of recorded moves

Result:

- Based on user input or computer algorithm input, updates board with selected piece moved

<u>void restartGame(void)</u>
Arguments: none
Result:

- Clears board and log so new game can start

<u>int checkMate(int playerNum, PIECE *board[RANKS][FILES])</u>
Arguments:

- playerNum: the current player, 0 for White and 1 for Black
- Board: current state of the game board

Result:

- If in a checkmate situation, function returns 1 or 2 (Depending on which player wins), else function returns 0.

<u>int staleMate(int playerNum, PIECE *board[RANKS][FILES])</u>
Arguments:

- int playerNum: integer denoting which player this function regards
- PIECE *board[RANKS][FILES]: board reference to pass game data

Result:

If the player has no possible moves, the function returns 1. If not, 0.

<u>int inCheck(int playerNum, PIECE *board[RANKS][FILES])</u>
Arguments:

Int playerNum: denotes which player this function is being applied to
PIECE *board[RANK][FILES]: Board reference to pass game data

Result:

Analyzes each of the opponents pieces and whether they have a possible move on the respective player's king's position. Returns 1 if possible and 0 if not.

Computer.c - Computer Player
Provides: move generation for the computer player
Requires: Chess.h, libRules.a, libSpecialRules.a, libStateGame.a

<u>MOVE *computerMove(PIECE *board[RANKS][FILES], int playerNum)</u>
Arguments:

- board: current state of the board

- playerNum: whose player it is, Black or White

Result:

- Provides a move of where the computer wants to go based off the current state of the board and calculating the best move

Logging.c - Tracking the move history

Provides: move log and file output for later viewing after the game has concluded

Requires: Chess.h

<u>int logMove(MOVE *m)</u>

Arguments:

- m: the move to be added to the move log

Result:

- Adds a new move entry to the log
- Returns 1 if the move is added successfully, 0 otherwise

<u>MOVELIST *createMoveList(MOVELIST *MoveList)</u>

Arguments:

- MoveList: pointer to the move list to be initialized

Result:

- Initializes an empty move list
- Returns a pointer to the initialized move list

<u>void deleteMoveList(MOVELIST *MoveList)</u>

Arguments:

- MoveList: pointer to the move list to be deleted

Result:

- Removes all entries from the move list
- Frees memory associated with the list

<u>void logToFile(const char *fileName, MOVELIST *MoveList)</u>

Arguments:

- filName: name of the output file
- MoveList: pointer to the move list containing recorded moves

Result:

- Writes the move log to the specific file in a human readable format

<u>void removeEntry(MOVELIST *MoveList, MENTRY *e)</u>

Arguments:

- MoveList: pointer to the list containing the entry
- e: pointer to the entry to be removed

Result:

- Removes the specific entry from the move list

- Updates the linked list connections

3.3 Detailed Description of Input and Output Formats

Syntax for Making a Move (Text-Based Version)

```
Select what piece you want to move: E2
Select where you want to go: E4
  +----+----+----+----+----+----+----+----+----+----+
8 | bR | bN | bB | bA | bQ | bK | bA | bB | bN | bR |
  +----+----+----+----+----+----+----+----+----+----+
7 | bP | bP | bP | bP | bP | bP | bP | bP | bP | bP |
  +----+----+----+----+----+----+----+----+----+----+
6 |    |    |    |    |    |    |    |    |    |    |
  +----+----+----+----+----+----+----+----+----+----+
5 |    |    |    |    |    |    |    |    |    |    |
  +----+----+----+----+----+----+----+----+----+----+
4 |    |    |    |    | wP |    |    |    |    |    |
  +----+----+----+----+----+----+----+----+----+----+
3 |    |    |    |    |    |    |    |    |    |    |
  +----+----+----+----+----+----+----+----+----+----+
2 | wP | wP | wP | wP |    | wP | wP | wP | wP | wP |
  +----+----+----+----+----+----+----+----+----+----+
1 | wR | wN | wB | wA | wQ | wK | wA | wB | wN | wR |
  +----+----+----+----+----+----+----+----+----+----+
    A    B    C    D    E    F    G    H    I    J
```

Format of Moves Recorded in User Log (Algebraic Notation)

```
White                          Black

1. e2-e4                       1. e7-e5

2. Kt g1-f3                    2. Kt b8-c6

3. B f1-b5                     3. a7-a6

4. B b5-a4                     4. Kt g8-f6

5. o-o                         5. b7-b5

6. B a4-b3                     6. B f8-e7

7. d2-d4                       7. d7-d6

8. c2-c3                       8. B c8-g4

9. B c1-e3                     9. o-o
```

4. Development Timeline

4.1 Partitioning of Tasks

Week 1	Week 2	Week 3	Week 4	Week 5
Create user manual	Create software specification Divide work amongst team members	Create alpha version of code with text-based interface	Fix any bugs with alpha version Add additional complex functionalities Convert text-based interface of alpha version to a GUI	Complete testing and any other troubleshooting

4.2 Team Member Responsibilities

Sanaa Bebal - Logging.h / Logging.c
Ava Chinn - User.h / User.c
Alonso De La Cruz - StateGame.h / StateGame.c
Estrella Distancia Zamora - Rules.h / Rules.c
Lauren Howe - SpecialRules.h / SpecialRules.c
Samuel Kim - Computer.h / Computer.c
David Lee - Chess.h / Chess.c

Note: All team members will contribute to the User Manual, Software Specification, Main.c, and the Makefile

5. References and External Libraries

https://github.com/healeycodes/andoma/tree/main/test
Referenced for inspiration for computer algorithm for determining the best move

https://canvas.eee.uci.edu/courses/81045/pages/projects
Used as a reference for the rules of anteater chess and for some setup and configuration instructions

https://github.com/jmqueen01/EECS22L-chess/blob/main/README
Used as a reference for installation Linux code

Contact Information

If you have any questions, comments, or concerns about this software, please contact the creators using the following email addresses:

- sbebal@uci.edu
- amchinn@uci.edu
- alonsojd@uci.edu
- edistanc@uci.edu
- lehowe@uci.edu
- samuek16@uci.edu
- davijl18@uci.edu

Note: Any email pertaining to this software sent to one or more of the addresses above may be forwarded to any or all of the other emails above.

Terms and Conditions

Copyright

© 2026 Thoughtless Airheads. All rights reserved.

The unauthorized sale or distribution of this software is prohibited.

Disclaimer of Warranty

THIS SOFTWARE IS PROVIDED “AS IS” WITHOUT ANY EXPRESS OR IMPLIED WARRANTY OF ANY KIND.

Index

Anteater, 3-5
anteating(), 8, 18
Ant, 3
Binary Distribution, 12
Board, 3, 4, 7, 8, 15, 19
castling(), 4, 7, 8, 17
Check, 3
checkMate(), 6, 7, 9, 19
Chess.c, 7-9, 18
Compilation, 12
Computer.c, 9, 20
computerMove(), 7, 9, 20
convertToAlgebraic(), 10
createMoveList(), 7, 9, 20
deleteMoveList(), 7, 9, 21
Development Timeline, 22
enPassant(), 7, 8, 17
GNU Compiler Collection (gcc), 12
GNU make,12
inCheck(), 7-9, 20
Installation, 12
legalMove(), 6, 8, 9, 17
Logging.c, 9, 20
logMove(), 9, 20
makeMove(), 7, 9, 19
Main.c, 6, 15
MENTRY, 5
MOVE, 5
MOVELIST, 5
parseMove(), 6, 7
pawnPromotion(), 8, 18
PIECE, 5
PIECECOLOR, 5
PIECETYPE, 5
POS, 5
restartGame(), 7, 9, 19
Rules.c, 7, 17
setPiece(), 8, 18
setPos(), 8, 19
Setup and Configuration, 12
Software Architecture, 5
Source Distribution, 13
SpecialRules.c, 8, 9, 17
staleMate(), 6, 7, 9, 20
switchPlayer(), 6, 7, 15
System Requirements, 12
undoMove(), 6, 9, 19
User.c, 7, 8, 15